KOENIG

step forward



REST APIs with Spring Boot

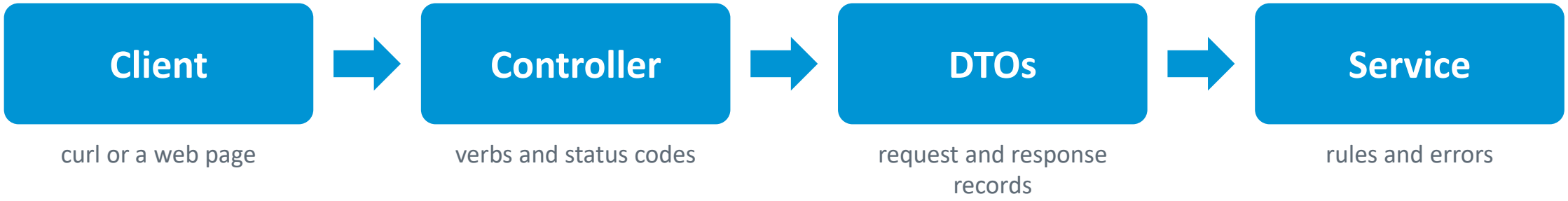
Spring Boot Fundamentals

What You Will Learn

1. REST resources, URIs, HTTP methods
2. Status codes and ResponseEntity
3. DTOs instead of entities
4. Validation with Jakarta Bean Validation
5. Central errors with ProblemDetail

Demo Scenario: Learnly Student REST API

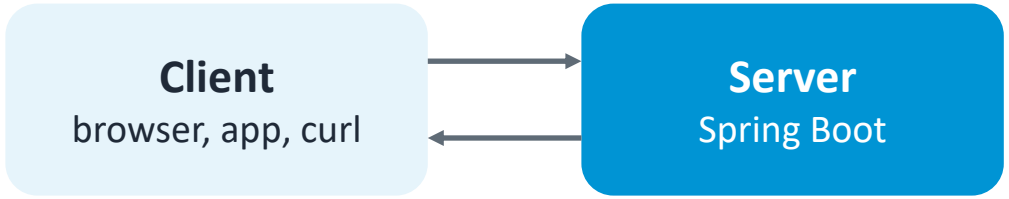
Module 7 exposed the Learnly courses. Now Learnly needs an API for its students, built properly: the right verbs and status codes, clean data shapes, validation, one error format, and cors.



Added step by step — **Validation** — **ProblemDetail** — **CORS**

What Is a REST API?

- **REST:** a style for web APIs built on plain HTTP.
- **Resources:** things like students, students, orders, named by URLs.
- **Verbs:** GET, POST, PUT, DELETE say what to do.
- **Stateless:** every request carries everything the server needs.
- **Format:** usually JSON.




```
GET /api/students/2 HTTP/1.1
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{"id":2,"name":"Bob","email":"bob@mail.com",
 "city":"Mumbai"}
```

Resources and URI Design

- **Nouns, plural:** /students, not /getStudents.
- **Identify one:** /students/42.
- **Nest for ownership:** /courses/3/students.
- **Filters go in the query:** ?city=Pune.
- **Lowercase, hyphens:** /student-reviews.

/students

/students/42

/students/42/reviews

/students?city=Pune&page=0

URI Design: Do and Do Not

Avoid

POST /createStudent

GET /getAllStudents

GET /students/delete/42

POST /students/42/update

/Student_List

Prefer

POST /students

GET /students

DELETE /students/42

PUT /students/42

/students

The URL names the thing, the verb names the action

HTTP Methods and Their Meaning

1 **GET**

Read. Never changes data.

GET /students/42

2 **POST**

Create a new resource.

POST /students

3 **PUT**

Replace a resource completely.

PUT /students/42

4 **DELETE**

Remove a resource.

DELETE /students/42

PATCH exists for partial updates; PUT sends the whole object

HTTP Status Codes You Need

200 OK
Success with a body

201 Created
New resource, add Location

204 No Content
Success, nothing to return

400 Bad Request
Invalid input

404 Not Found
No such resource

409 Conflict
Clashes with current state

415 / 405
Wrong content type / method

500 Server Error
Our bug

A REST Controller

- **@RestController** = @Controller + @ResponseBody.
- **@RequestMapping** sets the base path once.
- **@GetMapping etc.** map verb and sub-path.
- **Return value** is converted to JSON for us.

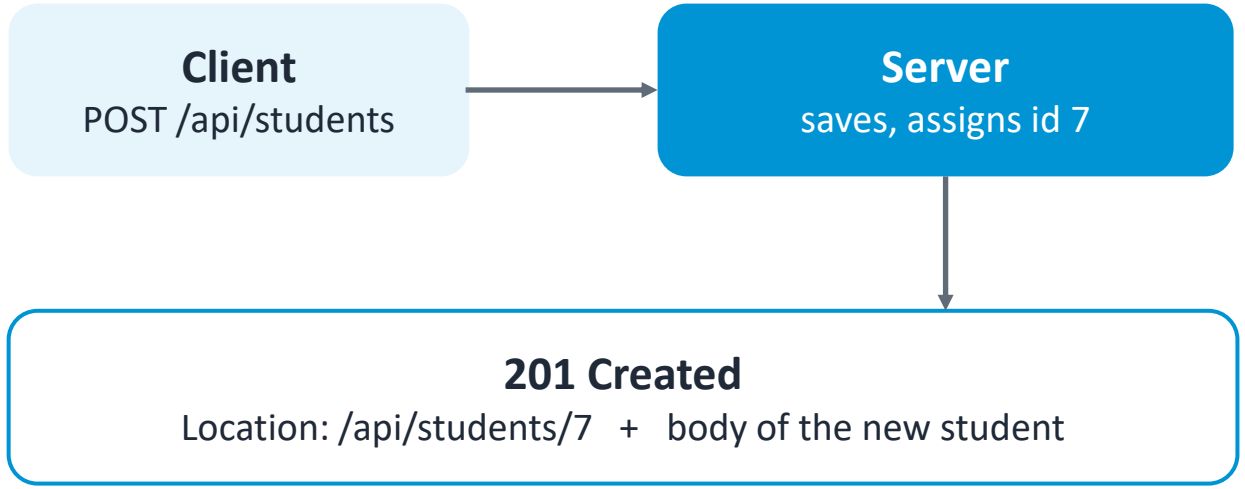
```
@RestController
@RequestMapping("/api/students")
public class StudentController {
    private final StudentService service;
    public StudentController(StudentService service) {
        this.service = service;
    }
    @GetMapping("/{id}")
    public StudentResponse get(@PathVariable long id) {
        return StudentMapper.toResponse(service.get(id));
    }
}
```

ResponseEntity: Full Control

- **Body:** what the client receives.
- **Status:** chosen by you.
- **Headers:** for example Location.
- **Shortcuts:** ok(), created(), noContent(), notFound().

```
@GetMapping("/{id}")
public ResponseEntity<Student> get(@PathVariable long id) {
    return service.find(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable long id) {
    if (!service.delete(id)) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.noContent().build();
}
```

201 Created and the Location Header




```
@PostMapping
public ResponseEntity<StudentResponse> create(
        @RequestBody StudentRequest request) {
    Student saved = service.create(
        StudentMapper.toEntity(request));
    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(saved.getId())
        .toUri();
    return ResponseEntity.created(location)
        .body(StudentMapper.toResponse(saved));
}
```

- **Location** tells the client where to find the new resource.
- **The server** creates the id, not the client.

Demo: Verbs, Status Codes and ResponseEntity

What we build

- **GET:** the list and one student, 404 when it is missing.
- **POST:** 201 Created with a Location header.
- **PUT and DELETE:** replace and remove, with 204 and 404.

How the concept helps

- **Verbs:** say what to do; the URL says to what.
- **Status codes:** let clients react without reading text.
- **ResponseEntity:** full control of status and headers.

Why Not Expose Entities?

Entity in the API

Every field leaks, including internal ones.

API and database change together.

Clients can send ids and flags they should not set.

Lazy JPA relations cause surprises.

DTOs in the API

You choose exactly what goes out and in.

Database can change without breaking clients.

Different shapes for create, update, read.

Validation lives on the request.

DTO = Data Transfer Object, the shipping label on the parcel

DTOs as Java Records

- **Records:** short, immutable, no boilerplate.
- **Request:** no id; the server creates it.
- **Response:** includes the id.
- **Jackson 3** reads and writes records directly.

```
public record StudentRequest(
        String name,
        String email,
        String city) {
}
public record StudentResponse(
        Long id,
        String name,
        String email,
        String city) {
}
```

Mapping Between DTO and Domain

- **Mapper class:** all conversion in one place.
- **Why not in the controller?** it stays short and testable.
- **Defaults:** fill in a value when the client sends none.
- **Later:** tools like MapStruct automate this.

```
public final class StudentMapper {
    public static Student toEntity(StudentRequest r) {
        Student student = new Student();
        student.setName(r.name());
        student.setEmail(r.email());
        student.setCity(r.city());
        return student;
    }
    public static StudentResponse toResponse(Student s) {
        return new StudentResponse(s.getId(), s.getName(),
            s.getEmail(), s.getCity());
    }
}
```

Demo: DTOs and a Mapper

What we build

- **The leak:** passwordHash shows up in the answer.
- **Records:** StudentRequest and StudentResponse.
- **StudentMapper:** converts in one place.

How the concept helps

- **Contract:** you choose what goes out and in.
- **Safety:** internal fields stay inside.
- **Freedom:** the database can change without breaking clients.

Bean Validation Basics

- **Jakarta Bean Validation:** rules as annotations.
- **Starter:** spring-boot-starter-validation.
- **Package:** jakarta.validation.constraints.
- **Message:** what the client reads.

```
public record StudentRequest(
    @NotBlank(message = "name is required")
    @Size(max = 60,
          message = "name must be at most 60 characters")
    String name,
    @NotBlank(message = "email is required")
    @Email(message = "email must be a valid address")
    String email,
    @NotBlank(message = "city is required")
    String city) {
}
```

Validation Annotations Cheat Sheet

@NotNull - not null

@NotBlank - text with a visible character

@NotEmpty - not null, not empty

@Size(min,max) - length of text or collection

@Min / @Max - number range

@Positive - greater than zero

@Email - email format

@Pattern(regexp) - matches a regex

@Future / @FutureOrPresent - date rules

@Past / @PastOrPresent - date rules

Triggering Validation with @Valid

- **@Valid** before @RequestBody switches checks on.
- **Without it** the annotations do nothing.
- **Failure** throws MethodArgumentNotValidException.
- **Result:** HTTP 400 before your code runs.

```
@PostMapping
public ResponseEntity<StudentResponse> create(
        @Valid @RequestBody StudentRequest request) {
    // only reached when the data is valid
    Student saved = service.create(
        StudentMapper.toEntity(request));
    ...
}
```

Demo: Bean Validation with @Valid

What we build

- **Constraints:** rules on the request record.
- **Without @Valid:** a bad student is saved.
- **With @Valid:** a 400 before our code runs.

How the concept helps

- **Rules:** sit next to the data they protect.
- **@Valid:** switches the checks on.
- **Early:** bad input never reaches the service.

Default Errors Are Not Enough

```
HTTP/1.1 400
Content-Type: application/json

{"timestamp":"2030-01-01T10:00:00",
 "status":400,
 "error":"Bad Request",
 "path":"/api/students"}
```

- **Problem:** which field is wrong? The client cannot tell.
- **Every API** invents its own error format.
- **Goal:** one standard, machine-readable shape.

Answer: RFC 9457 Problem Details

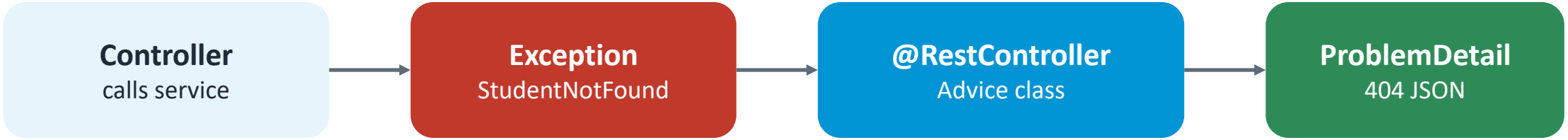
ProblemDetail (RFC 9457)

- **type:** URI naming the kind of problem (left out when it is about:blank).
- **title, status, detail:** human-readable summary.
- **instance:** the request path.
- **Extra properties:** e.g. errors.
- **Media type:** application/problem+json.

```
HTTP/1.1 400
Content-Type: application/problem+json

{
  "title": "Invalid request",
  "status": 400,
  "detail": "Validation failed",
  "instance": "/api/students",
  "errors": {
    "city": "city is required",
    "email": "email must be a valid address",
    "name": "name is required"
  }
}
```

Error Handling Flow



- **Services** throw domain exceptions; they know nothing about HTTP.
- **One advice class** maps exceptions to status and body.
- **Controllers** stay free of try/catch.

@RestControllerAdvice

- **@RestControllerAdvice:** global handler for all controllers.
- **@ExceptionHandler:** which exception to catch.
- **Return ProblemDetail** status and body in one object.
- **basePackages** can limit the scope.

```
@RestControllerAdvice
public class ApiErrorAdvice {

    @ExceptionHandler(StudentNotFoundException.class)
    public ProblemDetail notFound(StudentNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Student not found");
        return pd;
    }
}
```

Validation Errors as a Field Map

- **Extend** ResponseEntityExceptionHandler to cover all standard MVC errors.
- **Override** handleMethodArgumentNotValid.
- **Build** a map: field -> message.
- **Return** ProblemDetail with errors property.

```
@RestControllerAdvice
public class ApiErrorAdvice
        extends ResponseEntityExceptionHandler {

  @Override
  protected ResponseEntity<Object> handleMethodArgumentNotValid(
      MethodArgumentNotValidException ex, HttpHeaders headers,
      HttpStatusCode status, WebRequest request) {
    Map<String, String> errors = new TreeMap<>();
    for (FieldError e : ex.getBindingResult().getFieldErrors())
        errors.putIfAbsent(e.getField(), e.getDefaultMessage());
    ProblemDetail pd = ProblemDetail.forStatusAndDetail(
        HttpStatus.BAD_REQUEST, "Validation failed");
    pd.setProperty("errors", errors);
    return handleExceptionInternal(ex, pd, headers, status, request);
  }
}
```

Demo: One Error Shape with ProblemDetail

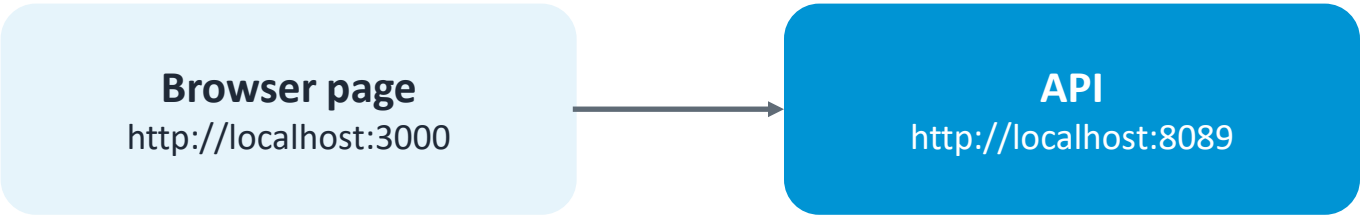
What we build

- **Exception:** StudentNotFoundException from the service.
- **Advice:** ApiErrorAdvice answers 404 as a ProblemDetail.
- **Field map:** validation errors, 405 and bad JSON too.

How the concept helps

- **One place:** every error answer is built here.
- **Standard:** RFC 9457, application/problem+json.
- **Clean code:** no try/catch in the controllers.

CORS: Cross-Origin Requests



- **Same-origin policy:** browsers block scripts calling another origin.
- **Origin** = scheme + host + port.
- **CORS:** the server says which origins are allowed.
- **Preflight:** OPTIONS request before PUT, DELETE or custom headers.
- **Not a server defence:** curl and Postman ignore CORS.

CORS Configuration in Spring

- **Global:** a WebMvcConfigurer bean.
- **Or per controller:** @CrossOrigin.
- **List origins** explicitly.
- **Avoid "*"** for APIs with cookies or tokens.

```java
@Configuration
public class CorsConfig {

  @Bean
  public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
      @Override
      public void addCorsMappings(CorsRegistry r) {
        r.addMapping("/api/**")
         .allowedOrigins("http://localhost:3000")
         .allowedMethods("GET","POST","PUT","DELETE")
         .allowedHeaders("Content-Type", "Accept")
         .maxAge(3600);
      }
    };
  }
}
```

Demo: CORS

What we build

- **CorsConfig:** allows http://localhost:3000.
- **Preflight from it:** 200 and the allow-origin header.
- **Preflight from another origin:** 403.

How the concept helps

- **Browser rule:** the server only says who may call.
- **Explicit origins:** list them, avoid *.
- **Not security:** curl and Postman ignore CORS.

Key Takeaways

1. **Resources and verbs**
   Nouns in URLs, HTTP verbs for actions, correct status codes.

2. **DTOs and validation**
   Records for the contract, @Valid for input checks.

3. **One error shape**
   @RestControllerAdvice and ProblemDetail everywhere.



Happy Learning :)